

LAB - CLOCK SYNCHRONIZATION

Student Details:

- **Name:** Mary Queenvine Muthoni
- **Admission Number:** 158716
- **Unit:** Distributed Systems
- **Title:** Clock Synchronization Using Berkeley's Algorithm
- **Group Number:** 14

Theory Berkeley's Algorithm is a clock synchronization technique utilized in distributed systems. The algorithm assumes that each machine node in the network either lacks an accurate time source or does not possess an external UTC server. In this architecture, an individual node is chosen as the master node from a pool of nodes in the network, while the rest of the nodes act as slaves. The master node periodically fetches the clock time from the slave nodes, calculates the average time difference between all the received clock times and its own system clock, and then broadcasts this adjusted time over the network so the slaves can synchronize.

Aim To develop an n-node distributed system that implements Berkeley's time synchronization algorithm.

Algorithm

1. A master node is chosen and a socket portal is opened to accept client connections over a designated port.
2. The master node periodically pings the slave clients to fetch their clock times.
3. The slave clients send their current local time to the master node.
4. The master node calculates the time difference between all received slave clocks and its own system clock.
5. The master calculates the average time difference of the network.
6. This average clock difference is added to the master's current system time to generate a synchronized time.
7. The master node sends this newly synchronized time back to the connected clients.
8. The slave nodes receive the synchronized time from the master and update their records.

Implementation

The lab was implemented using Python in a Linux environment running the Zsh shell. The system utilizes two separate files to establish the client-server synchronization framework:

- **Server.py** - acts as the master node, storing client address data, calculating average time differences, and distributing the synchronized time.

- Client.py - acts as the slave node, connecting to the master to send its local time and receive the corrected time.

(a) Software Requirements

- Python 3
- Python Virtual Environment (venv)
- python-dateutil library for parsing timestamps
- Linux Terminal (Zsh)

(b) Environment Setup & Compilation

To prevent PEP 668 externally-managed-environment errors, a virtual environment was created and activated within the lab directory, followed by the installation of the required parsing library.

```

Sun 19 Jul - 20:40 ~/Downloads/Clock Synchronization (1)
@user ~ pip install python-dateutil
Collecting python-dateutil
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 k
)
Collecting six>=1.5 (from python-dateutil)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, python-dateutil
Successfully installed python-dateutil-2.9.0.post0 six-1.17.0

```

(c) Execution - Server Deployment The server application was launched in a terminal window to host the master node. The server created a socket, bound to port 8080 on the local loopback address, and began listening for incoming connections.

```

Sun 19 Jul - 20:40 ~/Downloads/Clock Synchronization (1)
@user ~ python3 Server.py
Socket at master node created successfully

Clock server started...

Starting to make connections...

Starting synchronization parallelly...

New synchronization cycle started.
Number of clients to be synchronized: 0
No client data. Synchronization not applicable.

```

```
Client Data updated with: 127.0.0.1:58068

New synchronization cycle started.
Number of clients to be synchronized: 1

Client Data updated with: 127.0.0.1:58068

New synchronization cycle started.
Number of clients to be synchronized: 1

Client Data updated with: 127.0.0.1:58068

New synchronization cycle started.
Number of clients to be synchronized: 1
```

(d) Execution - Running the Clients In separate terminal windows, the client application was executed, connecting to the local server at 127.0.0.1 on port 8080.

```
Sun 19 Jul - 20:38 ~ /Downloads/Clock Synchronization (1)
@user source venv/bin/activate
python3 Client.py
Starting to receive time from server

Starting to receiving synchronized time from server

Recent time sent successfully

Synchronized time at the client is: 2026-07-19 20:41:09.063635

Synchronized time at the client is: 2026-07-19 20:42:04.075394

Recent time sent successfully

Synchronized time at the client is: 2026-07-19 20:42:09.076244

Recent time sent successfully

Synchronized time at the client is: 2026-07-19 20:42:14.076124
```

The master server continuously logged the updated client data, calculated the time variances, and successfully completed new synchronization cycles every 5 seconds.

Conclusion The Berkeley algorithm proved to be a simple yet effective solution for clock synchronization in distributed systems. By executing a central master server to compute average network time variances and distributing the corrected time back to the slave clients, the system effectively coordinated a unified time state across separate processes. The successful execution of the Python scripts verified that independent nodes can maintain accurate timekeeping without relying on external or centralized UTC servers.